By default, Spring Security stores the Access Tokens for the currently authenticated users in memory. When you have multiple instances of Edge Service running (which is always true in a cloud production environment to ensure high availability), you will encounter issues due to the statefulness of the application. Cloud native applications should be stateless. Let's fix that.

STORING ACCESS TOKENS IN REDIS

Spring Security stores Access Tokens in an `OAuth2AuthorizedClient` object that is accessible through a `ServerOAuth2AuthorizedClientRepository` bean. The default implementation for that repository adopts an in-memory strategy for persistence. That's what makes Edge Service a stateful application. How can we keep it stateless and scalable?

A simple way to do that is to store `OAuth2AuthorizedClient` objects in the web session rather than in memory so that Spring Session will pick them up automatically and save them in Redis, just like it does with ID Tokens. Fortunately, the framework already provides an implementation of the `ServerOAuth2AuthorizedClientRepository` interface to save data in the web session: `WebSessionServerOAuth2AuthorizedClientRepository`. Figure 12.3 illustrates how all the mentioned objects are related to each other.